

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



C.E.S.A.R.

Automação de Testes Web

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.



Testes de Automação Web

Automação Web
Aula 5



Estrutura do curso

Conteúdo:

Aula 1:

- Fundamentos da Automação de Testes
- Diferenças entre testes manuais e automatizados
- Estrutura de um Script de Teste
- Ferramentas

Aula 2:

- Identificação de Elementos
- Elementos dinâmicos e boas práticas
- Introdução ao padrão page object

Aula 3:

- Organização com page object
- Comandos, esperas e interações básicas
- Comandos, esperas e validações

Aula 4:

- Estrutura de projeto e boas práticas
- Modularização e reuso de código
- Construção de suite de testes

Aula 5:

- Criação de fluxo de teste completo
- Prática guiada
- Bonus: CI/CD + Paralelização

Intervalo: 15 à 20 minutos

Material das aulas: NEO

Avaliação Final: 5 questões objetivas

O que veremos hoje?

- Criação de fluxo de teste completo
- Bonus: Paralelização e CI/CD

Criação de fluxo de teste completo

1. login.feature (na pasta src/test/resources/features)

gherkin

Feature: Funcionalidade de login

Como um usuário do sistema

Quero fazer login com credenciais válidas

Para acessar a página inicial

Scenario: Login com sucesso

Given que o usuário está na página de login

When informa o usuário "admin" e a senha "123456"

And clica no botão de login

Then deve visualizar a página inicial

Criação de fluxo de teste completo

2. LoginPage.java (na pasta pages)

java

```
package pages;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class LoginPage {
```

```
    private WebDriver driver;
```

```
    private By campoUsuario = By.id("username");
```

```
    private By campoSenha = By.id("password");
```

```
    private By botaoLogin = By.id("loginButton");
```

```
    public LoginPage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
    public void acessarPaginaLogin() {
```

```
        driver.get("https://exemplo.com/login");
```

```
    }
```

```
    public void preencherUsuario(String usuario) {  
        driver.findElement(campoUsuario).sendKeys(usuario);  
    }
```

```
    public void preencherSenha(String senha) {  
        driver.findElement(campoSenha).sendKeys(senha);  
    }
```

```
    public void clicarLogin() {  
        driver.findElement(botaoLogin).click();  
    }
```

```
    public boolean estaNaPaginaInicial() {  
        return driver.getCurrentUrl().equals("https://exemplo.com/home");  
    }
```

Criação de fluxo de teste completo

3. LoginSteps.java (na pasta steps)

java

```
package steps;
```

```
import io.cucumber.java.en.*;
```

```
import org.junit.Assert;
```

```
import org.openqa.selenium.WebDriver;
```

```
import pages.LoginPage;
```

```
import support.DriverFactory;
```

```
public class LoginSteps {
```

```
    WebDriver driver = DriverFactory.getDriver();
```

```
    LoginPage loginPage = new LoginPage(driver);
```

```
    @Given("que o usuário está na página de login")
```

```
    public void queUsuarioEstaNaPaginaLogin() {
```

```
        loginPage.acessarPaginaLogin();
```

```
    }
```

```
    @When("informa o usuário {string} e a senha {string}")
```

```
    public void informaUsuarioSenha(String usuario, String senha) {
```

```
        loginPage.preencherUsuario(usuario);
```

```
        loginPage.preencherSenha(senha);
```

```
    }
```

```
    @And("clica no botão de login")
```

```
    public void clicaNoBotaoDeLogin() {
```

```
        loginPage.clicarLogin();
```

```
    }
```

```
    @Then("deve visualizar a página inicial")
```

```
    public void deveVisualizarPaginaInicial() {
```

```
        Assert.assertTrue(loginPage.estaNaPaginaInicial());
```

```
    }
```

```
}
```

INTERVALO!



20 min.

Paralelização e CI/CD

✓ O que é CI/CD?

Conceito	Significado
CI - Continuous Integration	Prática de integrar mudanças no código com frequência (várias vezes ao dia), rodando testes automaticamente para garantir estabilidade.
CD - Continuous Delivery/Deployment	Processo de entrega contínua de software — o código testado e aprovado é automaticamente entregue (ou implantado) em ambiente de homologação ou produção.

Paralelização e CI/CD

Benefícios de CI/CD em testes automatizados

Benefício	Explicação
Feedback rápido	Testes são executados a cada commit — erros são detectados cedo.
Padronização da execução	Evita "funciona na minha máquina".
Redução de trabalho manual	Execuções, builds e deploys automáticos.
Qualidade contínua	Garante que testes sempre rodem antes de liberar código novo.
Integração com relatórios	Relatórios de testes ficam disponíveis para toda a equipe.

Paralelização e CI/CD

Ferramentas comuns em CI/CD

Categoria	Exemplos
CI/CD	GitHub Actions, GitLab CI, Jenkins, CircleCI
Gerenciador de build	Maven, Gradle
Geração de relatório	Cucumber Reports, Allure, ExtentReports
Hospedagem de código	GitHub, GitLab, Bitbucket

Paralelização e CI/CD

1. Paralelização por Threads (nível de código)

- **Como funciona:** Configura múltiplos testes para rodarem em **threads separadas**, geralmente via `JUnit`, `TestNG` ou `Cucumber` com plugins.
- **Exemplo com Maven Surefire Plugin:**

xml

 Copy  Edit

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.0.0-M7</version>
  <configuration>
    <parallel>classes</parallel> <!-- ou methods, both -->
    <threadCount>4</threadCount>
  </configuration>
</plugin>
```

Paralelização e CI/CD

Considerações importantes

- Garanta que os testes são independentes (sem dependência de ordem ou estado).
- Evite uso de dados compartilhados entre threads.
- Use um gerenciador de sessão para garantir que cada teste use um navegador separado.
- Verifique se a infraestrutura da CI comporta recursos em paralelo (memória, CPU).

DÚVIDAS?!
:)





C . E . S . A . R

MUITO OBRIGADO!

© CESAR | Todos os direitos reservados